

Base Cases and Recursive Cases

The stack overflow example has a `shortest()` function that calls `shortest()` but never returns. To avoid a crash, there needs to be a case, or set of circumstances, where the function stops calling itself and instead just returns. This is called a *base case*. By contrast, a case where the function recursively calls itself is called a *recursive case*.

All recursive functions require at least one base case and at least one recursive case. If there is no base case, the function never stops making recursive calls and eventually causes a stack overflow. If there is no recursive case, the function never calls itself and is an ordinary function, not a recursive one. When you start writing your own recursive functions, a good first step is to figure out what the base case and recursive case should be.

Take a look at this *shortestWithBaseCase.py* program, which defines the shortest recursive function that won't crash from a stack overflow:

Python

```
def shortestWithBaseCase(makeRecursiveCall):
    print('shortestWithBaseCase(%s) called.' % makeRecursiveCall)
    if not makeRecursiveCall:
        # BASE CASE
        print('Returning from base case.')
        ❶ return
    else:
        # RECURSIVE CASE
        ❷ shortestWithBaseCase(False)
        print('Returning from recursive case.')
        return

    print('Calling shortestWithBaseCase(False):')
    ❸ shortestWithBaseCase(False)
    print()
    print('Calling shortestWithBaseCase(True):')
    ❹ shortestWithBaseCase(True)
```
